

程序设计 II 第 4 周 题目汇总（完整版）

来源：Matrix·课程「程序设计 II（25 级 曾坤 计算机学院）」含：完整题面、提交文件、平台提供的初始文件（.hpp/main.cpp 等）。选择题含标准答案。

共 7 项

25 程设 II-周 4-课后 1

类型：编程题

题面

字符串变形器

给定字符串，还有变形行数，输出二维 Z 形的字符串。构造一个 TRANSFORMER 类，在构造函数里输出"START TRANSFORMER"，在析构函数里输出"CLOSE TRANSFORMER"。由成员函数：printString(string str, int rowCount)进行字符串输出处理。

输入数据

第一行，字符串 s , $4 \leq \text{len}(s) \leq 300$ ；第二行，输出行数 r , $2 \leq r \leq 30$ ，且保证 $r \leq \text{len}(s)$ 。

输出数据

二维 z 字形排列的字符串，注意空格的插入。

输入示例 1

```
THISISAEXAMPLE
4
```

输出示例 1

```
START TRANSFORMER
T   A   L
H  S E P E
I I X M
S   A
CLOSE TRANSFORMER
```

输入示例 2

```
ABCDEFGH I J K L N M O P Q R S T U V W X Y Z 1 2 3 4 5 6 7 8 9
8
```

输出示例 2

```
START TRANSFORMER
A      O      3
B      M P    2 4
C      N Q    1 5
D      L R    Z 6
E      K      S Y 7
F J      T X    8
G I      U W    9
H      V
CLOSE TRANSFORMER
```

提交文件：TranStr.cpp

平台提供的初始文件

文件 main.cpp：

```
#include <iostream>
#include <TranStr.hpp>
using namespace std;

int main() {
    string str;
    int rowCount;
    TRANSFORMER tf;
    cin >> str >> rowCount;
    tf.printString(str, rowCount);
    return 0;
}
```

文件 TranStr.hpp：

```
#include <string>
class TRANSFORMER {
public:
    TRANSFORMER();
    ~TRANSFORMER();
    void printString(std::string str, int rowCount);
};
```

25 程设 II-周 4-课后 2

类型：编程题

题面

数字转换

给定一个非负整数金额 num ($0 \leq num \leq 1e^{12} - 1$ ，也即小于一万亿)，输出中文大写的金额。

提示：大写的中文数字分别为：

"零", "壹", "贰", "叁", "肆", "伍", "陆", "柒", "捌", "玖"

大写的中文单位为：

"拾", "佰", "仟", "万", "亿"

特别需要注意转换后“零”的使用。

输入数据

第一行，一个正整数，且保证小于一万亿。

输出数据

中文大写的金额，注意结尾为“元整”，可以参考 <https://tool.lu/daxie/> 的转换工具来测试验证。

输入示例 1

1

输出示例 1

壹元整

输入示例 2

10101001

输出示例 2

壹仟零壹拾万零壹仟零壹元整

输入示例 3

123456789876

输出示例 3

壹仟贰佰叁拾肆亿伍仟陆佰柒拾捌万玖仟捌佰柒拾陆元整

输入示例 4

100010001

输出示例 4

壹亿零壹万零壹元整

提交文件：main.cpp

(本题无平台初始文件)

25 程设 II-周 4-课后 3

类型：编程题

题面

颜色类

请实现一个表示颜色的类 ColorRGB。具体功能要求如下：

公有数据成员：r（红）、g（绿）、b（蓝），均为 float 类型，取值范围暂不作限制。

构造函数：

ColorRGB(float r, float g, float b); // 使用给定的 r, g, b 分量创建颜色。

静态数据成员（提供常用的标准颜色）：

- black: 代表纯黑 (0.0, 0.0, 0.0)。
- white: 代表纯白 (1.0, 1.0, 1.0)。
- red: 代表纯红 (1.0, 0.0, 0.0)。

静态函数成员：

- Distance(ColorRGB c1, ColorRGB c2): 计算并返回两个颜色在 RGB 空间中的欧式距离（计算公式与三维坐标距离的计算公式相同）。
- Mix(ColorRGB c1, ColorRGB c2): 将两种颜色混合，返回一个新的颜色。混合规则为两个颜色的 r, g, b 分量分别相加后除以 2。
- GetBrightness(ColorRGB c): 计算并返回颜色的平均亮度，公式为： $(r+g+b)/3.0$ 。

输入说明

输入三个浮点数，代表一个自定义颜色 c 的 r, g, b 分量。

0.2 0.4 0.6

输出说明

分别按行输出以下 3 项计算结果（保留 1 位小数，不需要任何文字描述）：

1. 颜色 c 与静态颜色 white 的距离 (Distance)。

2. 颜色 c 与静态颜色 red 混合 (Mix) 后的新颜色的 r, g, b 分量 (用空格隔开)。
 3. 颜色 c 的平均亮度 (GetBrightness)。
- 1.1
0.6 0.2 0.3
0.4

提交文件：ColorRGB.cpp

平台提供的初始文件

文件 main.cpp：

```
#include <iostream>
#include <iomanip>
#include "ColorRGB.hpp"

using namespace std;

int main() {
    float r, g, b;

    if (cin >> r >> g >> b) {
        ColorRGB c(r, g, b);

        cout << fixed << setprecision(1);

        // 1. 与 white 的距离
        cout << ColorRGB::Distance(c, ColorRGB::white) << endl;

        // 2. 与 red 混合后的分量
        ColorRGB mixed = ColorRGB::Mix(c, ColorRGB::red);
        cout << mixed.r << " " << mixed.g << " " << mixed.b << endl;

        // 3. 计算亮度
        cout << ColorRGB::GetBrightness(c) << endl;
    }

    return 0;
}
```

文件 ColorRGB.hpp：

```
#pragma once

class ColorRGB {
public:
    float r;
    float g;
    float b;
```

```
ColorRGB(float r, float g, float b);

// 静态数据成员申明
static const ColorRGB black;
static const ColorRGB white;
static const ColorRGB red;

// 静态成员函数申明(传值)
static float Distance(ColorRGB c1, ColorRGB c2);
static ColorRGB Mix(ColorRGB c1, ColorRGB c2);
static float GetBrightness(ColorRGB c);
};
```

25 程设 II-周 4-课后 4

类型：编程题

题面

简单栈实现：

实现一个简易的栈（Stack）类，要求如下：

- 栈使用固定大小的数组 `datas[20]` 存储数据。
- `tail` 变量表示栈顶元素的下一个位置。初始时，栈为空，`tail` 为 0。
- 需要实现以下成员函数：
 - `stack()`：构造函数，初始化栈为空栈。构造函数执行时，需要输出 “Stack constructor”。
 - `~Stack()`：析构函数，栈销毁时输出 “Stack destructor”。
 - `push(int)`：将一个整数压入栈顶。如果栈已满，输出 “Stack is full!”。
 - `pop()`：移除栈顶元素。如果栈为空，输出 “Stack is empty!”。
 - `top() const`：返回栈顶元素的值。如果栈为空，输出 “Stack is empty!” 并返回 -1。
 - `empty() const`：判断栈是否为空，如果为空返回 `true`，否则返回 `false`。
 - `clear()`：清空栈，将 `tail` 置为 0。

输入数据

无

输出数据

开始栈测试...
Stack constructor

--- 基本测试 ---

```
栈是否为空 (1=是, 0=否): 1
栈是否为空: 0
栈顶元素: 20
栈顶元素: 10
栈是否为空: 1
Stack is empty!
栈顶元素: Stack is empty!
-1
```

--- 填满栈的测试 ---

```
栈是否已满: 1
Stack is full!
```

--- 清空栈并再次测试 ---

```
栈是否为空: 1
栈顶元素: 5
栈是否为空: 1
尝试从空栈取栈顶: Stack is empty!
-1
```

测试完成.
Stack destructor

提交文件 : Stack.cpp

平台提供的初始文件

文件 main.cpp :

```
#include <iostream>
#include "Stack.h"
```

```
using namespace std;
```

```
int main() {
    cout << "开始栈测试..." << endl;
    Stack s;

    // 基本测试
    cout << "\n--- 基本测试 ---" << endl;
    cout << "栈是否为空 (1=是, 0=否): " << s.empty() << endl; // 预期1
    s.push(10);
    cout << "栈是否为空: " << s.empty() << endl; // 预期0
    s.push(20);
    cout << "栈顶元素: " << s.top() << endl; // 预期20
    s.pop();
    cout << "栈顶元素: " << s.top() << endl; // 预期10
    s.clear();
    cout << "栈是否为空: " << s.empty() << endl; // 预期1
    s.pop(); // 尝试从空栈pop, 预期输出"Stack is empty!"
```

```

    cout << "栈顶元素: " << s.top() << endl; // 尝试从空栈获取top，预期输出"Stack is
empty!"，并返回-1

    // 填满栈的测试
    cout << "\n--- 填满栈的测试 ---" << endl;
    for (int i = 1; i <= 20; ++i) {
        s.push(i * 5);
    }
    cout << "栈是否已满: " << (s.top() == 100) << endl; //通过最后一个压入的元素判断
栈是否已满，预期1
    s.push(101); // 尝试push 超过栈容量的元素，预期输出"Stack is full!"

    // 清空栈并再次测试
    cout << "\n--- 清空栈并再次测试 ---" << endl;
    s.clear();
    cout << "栈是否为空: " << s.empty() << endl; // 预期1
    s.push(5);
    cout << "栈顶元素: " << s.top() << endl; //预期5
    s.pop();
    cout << "栈是否为空: " << s.empty() << endl; //预期1
    cout << "尝试从空栈取栈顶:" << s.top() << endl;

    cout << "\n 测试完成." << endl;

    return 0;
}

```

文件 Stack.h :

```

#ifndef STACK_H
#define STACK_H

#include <iostream>

class Stack {
private:
    int tail;
    int datas[20];

public:
    Stack();
    ~Stack();
    void push(int val);
    void pop();
    int top() const;
    bool empty() const;
    void clear();
};

```


#endif

25 程设 II-周 4-课堂 1

类型：编程题

题面

月份天数

输入年份和月份，给出该月的天数

输入数据

第一行，年份 y , $1900 \leq y \leq 3000$ ；第二行，月份 m , $1 \leq m \leq 12$ 。

输出数据

该年该月的天数。**提示**：闰年的二月为 29 天，公元年数能被 4 整除的年为闰年，其余年为平年。但对整世纪年（如 1600，1700.....）只有世纪数也能被 4 整除的才是闰年，其余的仍为平年，比如 2000 年是闰年，而 1900 年不是。

输入示例 1

2022
1

输出示例 1

31

输入示例 2

2020
2

输出示例 2

29

输入示例 3

1900
2

输出示例 3

28

提交文件：monthDay.cpp

平台提供的初始文件

文件 main.cpp：

```
#include <iostream>
#include <monthDay.hpp>

int main() {
    int year, month;
    std::cin >> year >> month;
    MonthDay m_day(year, month);
    return 0;
}
```

文件 monthDay.hpp：

```
class MonthDay {
private:
    int year, month;
public:
    MonthDay(int y, int m) {
        year = y;
        month = m;
    }
    ~MonthDay();
};
```

25 程设 II-周 4-课堂 2

类型：编程题

题面

矩形盒子

请设计一个名为 Box 的类，要求如下：

1. 包含三个私有数据成员：length、width、height，类型均为 double。
2. 提供一个带有默认实参的构造函数，当不提供参数时，默认长、宽、高均为 2.0。
3. 提供成员函数 double Volume() 计算并返回盒子的体积。
4. 提供成员函数 bool compare(Box box)，在函数内部必须使用 this 指针调用当前对象的 Volume() 方法，比较当前对象与传入对象 box 的体积。如果当前对象体积大于传入对象，返回 true，否则返回 false。

输入数据

输入一行，包含三个浮点数中间用空格分隔，分别代表 Box 的长宽高。

输入样例

1.0 2.0 3.0

输出数据

第一行输出由输入数据构成的 Box 的体积。第二行，如果输入的 Box 体积大于长、宽、高都为 2 的 Box，则输出 yes，否则输出 no。

输出样例

6
no

提交文件：Box.cpp、Box.hpp

平台提供的初始文件

文件 main.cpp：

```
#include <iostream>
#include "Box.hpp" // 引入自定义的类声明

using namespace std;

int main() {

    double l,w,h;

    cin >> l >> w >> h;

    Box Box1(l, w, h);
    Box Box2;
    cout << Box1.Volume() << endl;
    // 调用compare方法进行比较[cite: 435]
    if (Box1.compare(Box2)) {
        cout << "yes" << endl;
    } else {
        cout << "no" << endl;
    }
    return 0;
}
```

25 程设II-周 4-理论题

类型：选择题（20 题）

第 1 题

一个类可以有多少个构造函数

- A. 1
- B. 2
- C. 3
- D. 任意多个

答案：D

解析：一个类可以有任意多个构造函数，但一个类的实例能且只能调用一次构造函数

第 2 题

一个类可以有多少个析构函数

- A. 1
- B. 2
- C. 3
- D. 任意多个

答案：A

解析：本质上是因为析构函数不能传参，因此有且仅有一个析构函数签名

第 3 题

下列 C++ 代码的输出是什么

```
#include <iostream>
using namespace std;
class TEST {
    int a, b, c;
}
int main() {
    TEST test;
    cout << "In main()\n";
    return 0;
}
```

- A. In main()
- B. In main()
- C. 编译错误
- D. 段错误 (Segmentation Fault)

答案：c

解析：类的声明部分的{}之后应该用分号；结尾。

第 4 题

下列 C++ 代码的输出是什么

```
#include <iostream>
using namespace std;
class A{
private:
    int a, b, c;
public:
    A(int a, int b, int c) {
        this->a = a;
        this->b = b;
        this->c = c;
        cout << "A" << endl;
    }
    ~A() {
        cout << "~A" << endl;
    }
};
int main() {
    A test;
    return 0;
}
```

- A. A ~A
- B. A
- C. ~A
- D. 编译错误

答案：D

解析：显示声明了构造函数，因此编译器不会自动补上默认构造函数，因此编译报错：类“A”不存在默认构造函数

第 5 题

下列 C++ 代码的输出是什么

```
#include <iostream>
using namespace std;
class A{
    int a, b, c;
    A() {
        cout << "A1" << endl;
    }
    A(int a, int b, int c) {
```

```

        this->a = a;
        this->b = b;
        this->c = c;
        cout << "A2" << endl;
    }
    ~A() {
        cout << "~A" << endl;
    }
};
int main() {
    A test;
    return 0;
}

```

- A. A1 ~A
- B. A2 ~A
- C. A1 A2 ~A
- D. 编译错误

答案：D

解析：类的声明里没有显式访问控制时默认为 private，导致 main 函数无法访问默认构造函数，编译报错：“A::A()”不可访问

第 6 题

下列 C++ 代码的输出是什么

```

#include <iostream>
#include <string>
using namespace std;

class A{
    string name;
public:
    A(string name) {
        this->name = name;
        cout << "A(" << name << ")" << endl;
    }
    ~A() {
        cout << "~A(" << name << ")" << endl;
    }
};

int main() {
    A a1("a1"), a2("a2");
    return 0;
}

```

- A. A(a1) A(a2) ~A(a2) ~A(a1)

- B. A(a1) A(a2) ~A(a1) ~A(a2)
- C. A(a1) ~A(a1) A(a2) ~A(a2)
- D. 编译错误

答案：A

第 7 题

下列 C++ 代码的输出是什么

```
#include <iostream>
using namespace std;

class A{
    string name;
public:
    A(string name) {
        this->name = name;
        cout << "A(" << name << ")" << endl;
    }
    ~A() {
        cout << "~A(" << name << ")" << endl;
    }
};

int main() {
    A a1("a1");
    { A a2("a2"); }
    A a3("a3");
    return 0;
}
```

- A. A(a1) A(a2) A(a3) ~A(a2) ~A(a3) ~A(a1)
- B. A(a1) A(a2) ~A(a2) A(a3) ~A(a3) ~A(a1)
- C. A(a1) ~A(a1) A(a2) ~A(a2) A(a3) ~A(a3)
- D. 编译错误

答案：B

解析：{} 的出现影响了 a2 的生命周期，使其提前被回收，因此提前调用析构函数

第 8 题

下列 C++ 代码的输出是什么

```
#include <iostream>
using namespace std;

class A{
```

```

    int a = 0;
    int b = 0;
    int c = 0;
public:
    A(int a, int b = 1, int c = 1) {
        this->a = a;
        this->b = b;
    }
    ~A() {
        cout << a << b << c << endl;
    }
};

int main() {
    A a(2,2,2);
    return 0;
}

```

- A. 000
- B. 222
- C. 220
- D. 211

答案：c

第 9 题

下列 C++ 代码的输出是什么

```

#include <iostream>
using namespace std;

class A{
    int a = 0;
    int b = 0;
    int c = 0;
public:
    A(int a, int b = 1, int c = 1){
        this->a = a;
        this->b = b;
        this->c = c;
    }
    ~A() {
        cout << a << b << c << endl;
    }
};

int main() {
    A a(2,2,2);
}

```



```
    return 0;
}
```

- A. 221
- B. 222
- C. 000
- D. 111

答案：B

第 10 题

下列 C++ 代码的输出是什么

```
#include <iostream>
using namespace std;

class A{
    int a = 0;
    int b = 0;
    int c = 0;
public:
    A(int a, int b = 1, int c = 1) {
        this->a = a;
        this->b = b;
        this->c = c;
    }
    ~A() {
        cout << a << b << c << endl;
    }
};

int main() {
    A a(3);
    return 0;
}
```

- A. 111
- B. 331
- C. 313
- D. 311

答案：D

第 11 题

以下关于构造函数的描述，错误的是？

- A. 构造函数的名称必须与类名相同
- B. 构造函数不能有返回值类型，包括 void

- C. 构造函数不能重载
- D. 构造函数可以有默认参数

答案：C

解析：构造函数是可以重载的

第 12 题

下面关于析构函数的说法，正确的是？

- A. 析构函数必须由程序员手动调用
- B. 如果一个类没有析构函数，编译器不会生成默认析构函数
- C. 析构函数通常用于释放动态分配的内存
- D. 析构函数不能为 private

答案：C

第 13 题

下面哪个函数不能访问 this 指针？

- A. 普通成员函数
- B. 构造函数
- C. 虚函数
- D. 静态成员函数

答案：D

解析：静态成员函数不依赖对象

第 14 题

以下关于静态成员变量的描述，错误的是？

- A. 静态成员变量不能是 const
- B. 静态成员变量必须在类外初始化
- C. 静态成员变量不能在构造函数内初始化
- D. 静态成员变量在所有对象间共享

答案：A

解析：静态成员变量可以是 const

第 15 题

类的构造函数的作用是什么？

- A. 必要时修改数据成员
- B. 销毁一个对象

- C. 产生该对象时，初始化其数据成员
- D. 在外部调用私有成员函数

答案：C

解析：拷贝构造函数参数必须是引用，否则会递归调用自身

第 16 题

下列 C++ 代码的输出是什么

```
#include <iostream>
using namespace std;
class A {
public:
    A() { cout << "A "; }
    ~A() { cout << "~A "; }
};
void func() {
    static A a;
}
int main() {
    func();
    func();
    return 0;
}
```

- A. A ~A A ~A
- B. A ~A
- C. A A ~A ~A
- D. 编译错误

答案：B

解析：static 对象只初始化一次，析构函数在程序结束时调用

第 17 题

下列哪个选项是程序的输出：

```
class CDummy
{
public:
    int isitme (CDummy& param);
};

int CDummy::isitme (CDummy& param)
{
    if (&param == this)
        return true;
    else
```

```

        return false;
    }

int main ()
{
    CDummy a;
    CDummy *b = &a;
    if (b->isitme(a)) {
        cout << "execute";
    }
    else
    {
        cout<<"not execute";
    }
    return 0;
}

```

- A. “execute”
- B. “not execute”
- C. 以上都不是
- D. 0

答案：A

解析：传值调用拷贝构造，返回值拷贝

第 18 题

下列 C++ 代码的输出是什么

```

#include <iostream>
using namespace std;
class A {
public:
    static int count;
    A() { count++; }
    ~A() { count--; }
};
int A::count = 0;
int main() {
    A a1, a2;
    cout << A::count << endl;
}

```

- A. 1
- B. 2
- C. 0
- D. 编译错误

答案：B

解析：count 记录了对象数目，还未调用析构函数

第 19 题

下列 C++ 代码的输出是什么

```
#include <iostream>
using namespace std;

class A {
public:
    int x;
    A(int val){
        x=val;
    }

    A& setX(int x) {
        this->x = x * 2;
        return *this;
    }

    void print() const {
        cout << "x = " << this->x << endl;
    }
};

int main() {
    A obj1(5);
    A obj2(10);

    obj1.setX(obj2.x).setX(3);

    obj1.print();
    obj2.print();

    return 0;
}
```

- A. x = 6 x = 10
- B. x = 10 x = 10
- C. x = 6 x = 6
- D. x = 3 x = 10

答案：A

第 20 题

下列 C++ 代码的输出是什么

```

#include <iostream>
using namespace std;

class MyClass {
private:
    int id;
    static int count; // 静态成员变量

public:
    MyClass(int id){
        this->id=id;
        count++;
        cout << "Constructor: Object " << this->id << " created. Total objects: " <<
count << endl;
    }

    ~MyClass() {
        count--;
        cout << "Destructor: Object " << this->id << " destroyed. Total objects: " <<
count << endl;
    }

    void showId() const {
        cout << "Object ID: " << this->id << endl;
    }

    static int getCount() { // 静态成员函数
        return count;
    }
};

int MyClass::count = 0; // 初始化静态成员变量

int main() {
    MyClass obj1(1);
    {
        MyClass obj2(2);
        obj2.showId();
    }
    cout << "Total objects after block: " << MyClass::getCount() << endl;
    return 0;
}

```

- A. ...Total objects after block: 1 / Destructor: Object 1 ... Total objects: 0
- B. ...Total objects after block: 0...
- C. ...Total objects after block: 2...
- D. ...Total objects: -1

答案：A
